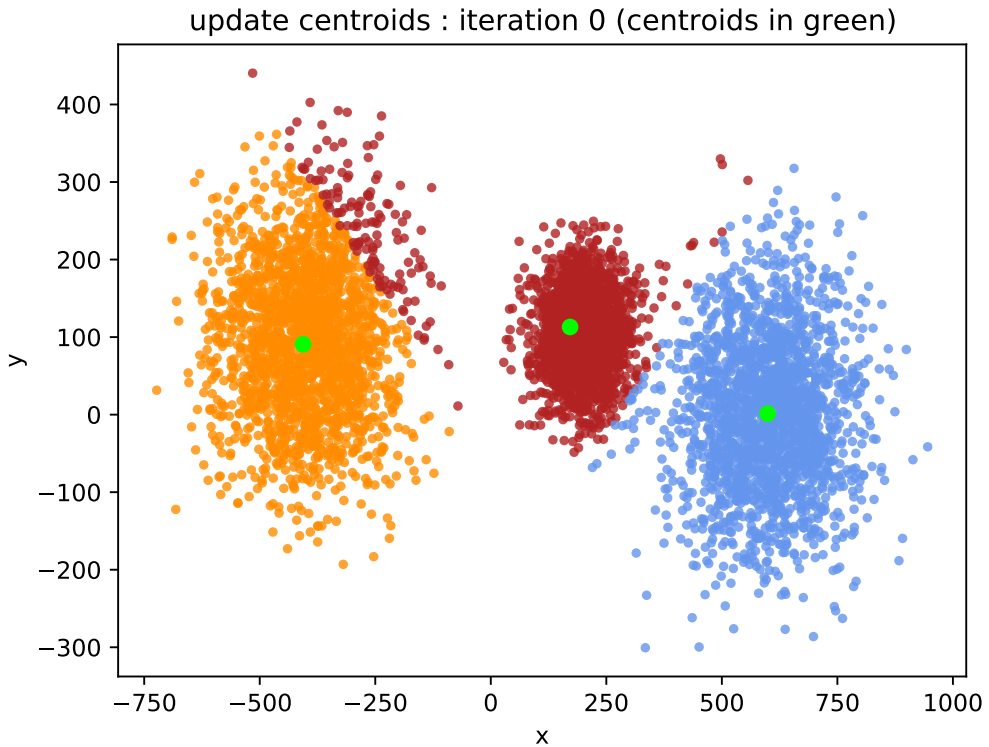
Fondamentaux théoriques du machine learning



Dimensionality reduction

Principal component analysis

Clustering

Dimensionality reduction

We consider the space $\mathcal{X}$ that contains the data, for either supervised or unsupervised learning. In machine learning, we often have $\mathcal{X} \in \mathbb{R}^d$.

- If d is large (e.g. $\geq 10^4$), the algorithms that run on the data might become too slow to be used, as their algorithmic complexity depends on d (potentially in a quadratic or exponential way, curse of dimensionality)

Dimensionality reduction

We consider the space $\mathcal{X}$ that contains the data, for either supervised or unsupervised learning. In machine learning, we often have $\mathcal{X} \in \mathbb{R}^d$.

- If d is large (e.g. $\geq 10^4$), the algorithms that run on the data might become too slow to be used, as their algorithmic complexity depends on d (potentially in a quadratic or exponential way, curse of dimensionality)
- **However**, often the data might actually occupy a **subspace** of lower dimension q, or it may be possible to project the data on such a subspace without loosing too much information.
  - Working in a subspace of lower dimension might speed up the algorithms.
  - It may also allow visualization of the data.

Main methods of dimensionality reduction

- **feature selection** : selecting a subset of the original dimensions.
- **feature extraction** : computing new features from the original features.

Principal component analysis (PCA)

- PCA is a **linear feature extraction** technique.
- Points in $\mathbb{R}^d$ are linearly projected on a well chosen affine subspace of $\mathbb{R}^q$, with $q \leq d$.

Formalization as a (empirical) variance maximimisation problem

Without loss of generality, we assume the data are **centered**, which means that

$$\bar{x} = \sum_{i=1}^{n} x_n = 0 \in \mathbb{R}^d \tag{1}$$

X is the design matrix like in OLS. The **first principal componenent** is a vector $w \in \mathbb{R}^d$, with $||w|| = 1$, such that $\hat{Var}(w^T x)$ is maximal, where $\hat{Var}$ denotes the empirical variance.

Variance

$$\overline{w^T x} = w^T \overline{x} = 0 \tag{2}$$

Hence,

$$\begin{aligned} \hat{Var}(w^T x) &= \frac{1}{n-1} \sum_{i=1}^{n} \left((w^T x)_i - \overline{w^T x} \right)^2 \\ &= \frac{1}{n-1} \sum_{i=1}^{n} \left(w^T x_i \right)^2 \end{aligned} \tag{3}$$

Variance maximisation problem

We can then formulate the problem as finding w, $||w|| = 1$ such that

$$\sum_{i=1}^{n} \left(w^T x_i\right)^2 \tag{4}$$

is maximal.

First principal component

We look for w, $||w|| = 1$ such that

$$\sum_{i=1}^{n} \left(w^T x_i\right)^2 \tag{5}$$

is maximal.

Proposition

w is the eigenvector of $X^T X$ with largest eigenvalue λ_{max}.

First principal component

We look for w, $||w|| = 1$ such that

$$\sum_{i=1}^{n} \left(w^T x_i\right)^2 \tag{6}$$

is maximal.

Proposition

w is the eigenvector of $X^T X$ with largest eigenvalue λ_{max}.

Exercice 1 : Show the proposition.

Reconstruction error

Alternately, we can formulate the problem as a **reconstruction error minimization.**

$$\mathbb{R}^d = \text{Vect}(w) \bigoplus \text{Vect}(w)^{\perp} \tag{7}$$

and if $||w|| = 1$,

$$\begin{aligned} \forall x \in \mathbb{R}^d, ||x||^2 &= ||(x^T w)w||^2 + ||x - (x^T w)w||^2 \\ &= (x^T w)^2 + ||x - (x^T w)w||^2 \end{aligned} \tag{8}$$

Reconstruction error

We can formulate the problem as a **reconstruction error minimization.** If $||w|| = 1$,

$$\begin{aligned} \forall x \in \mathbb{R}^d, ||x||^2 &= ||(x^T w)w||^2 + ||x - (x^T w)w||^2 \\ &= (x^T w)^2 + ||x - (x^T w)w||^2 \end{aligned} \tag{9}$$

Hence,

$$\begin{aligned} \sum_{i=1}^{n} ||x_i||^2 &= \sum_{i=1}^{n} (x_i{}^T w)^2 + \sum_{i=1}^{n} ||x_i - (x_i{}^T w)w||^2 \\ &= \hat{Var}(w^T x) + \sum_{i=1}^{n} ||x_i - (x_i{}^T w)w||^2 \end{aligned} \tag{10}$$

Reconstruction error

$$\sum_{i=1}^{n} ||x_i||^2 = \hat{Var}(w^T x) + \sum_{i=1}^{n} ||x_i - (x_i^T w)w||^2 \qquad (11)$$

We can see $\sum_{i=1}^{n} ||x_i - (x_i^T w)w||^2$ as a **reconstruction error**, when the data are projected on Vect(w).
Maximizing the variance of the projections is equivalent to minimizing the reconstruction errors obtained by projection.

Several principal components

Most of the time, we project the data on several principal components.

- 1] compute the first principal component w_1
- 2] project the data on $\text{Vect}(w_1)^\perp$
- 3] start again on the projected data

Reconstruction error

The interpretation stays the same. If $p_F(x)$ is the projection of x on the subspace spanned by the principal components :

$$||x||^2 = ||p_F(x)||^2 + ||x - p_F(x)||^2 \tag{12}$$

The principal components are the largest eigenvectors of $X^T X \in \mathbb{R}^d, d$ with norm 1. They are **orthogonal** to each other.

Inertia

We can define an inertia :

$$I_F = \sum_{i=1}^{n} ||x - p_F(x)||^2 \tag{13}$$

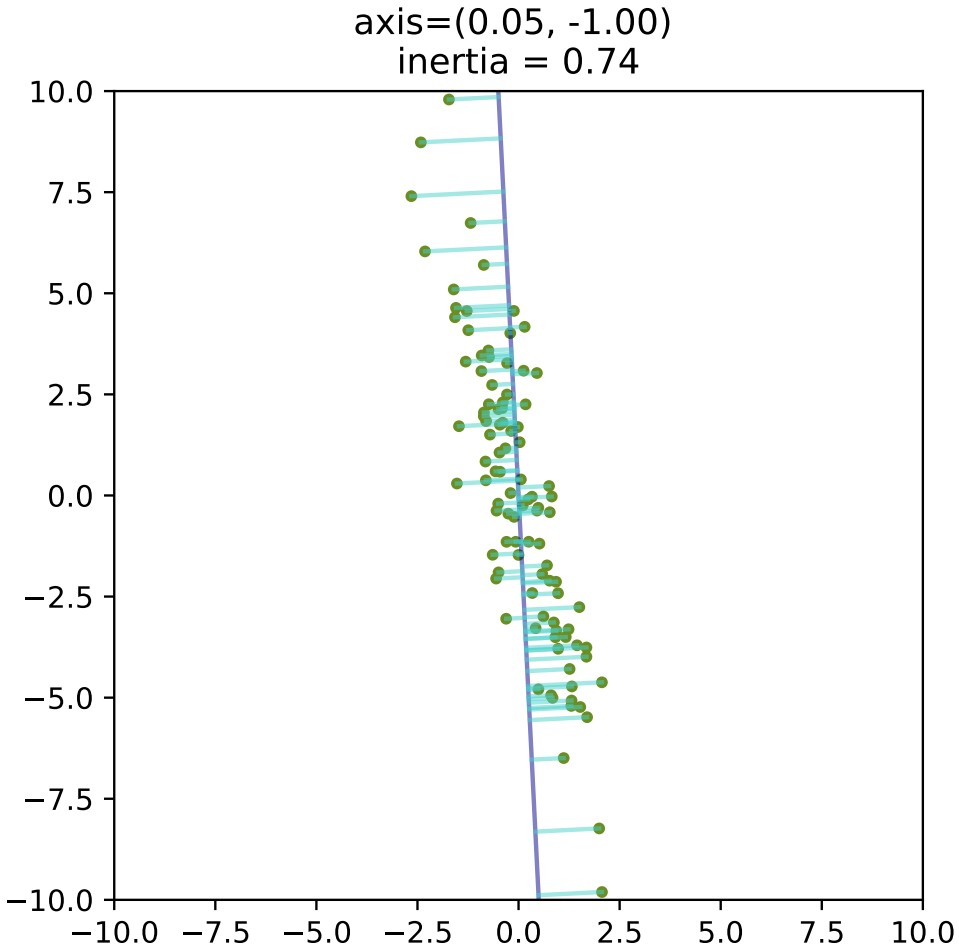
We lool for the subspace that minimizes the inertia I_F.

Inertia

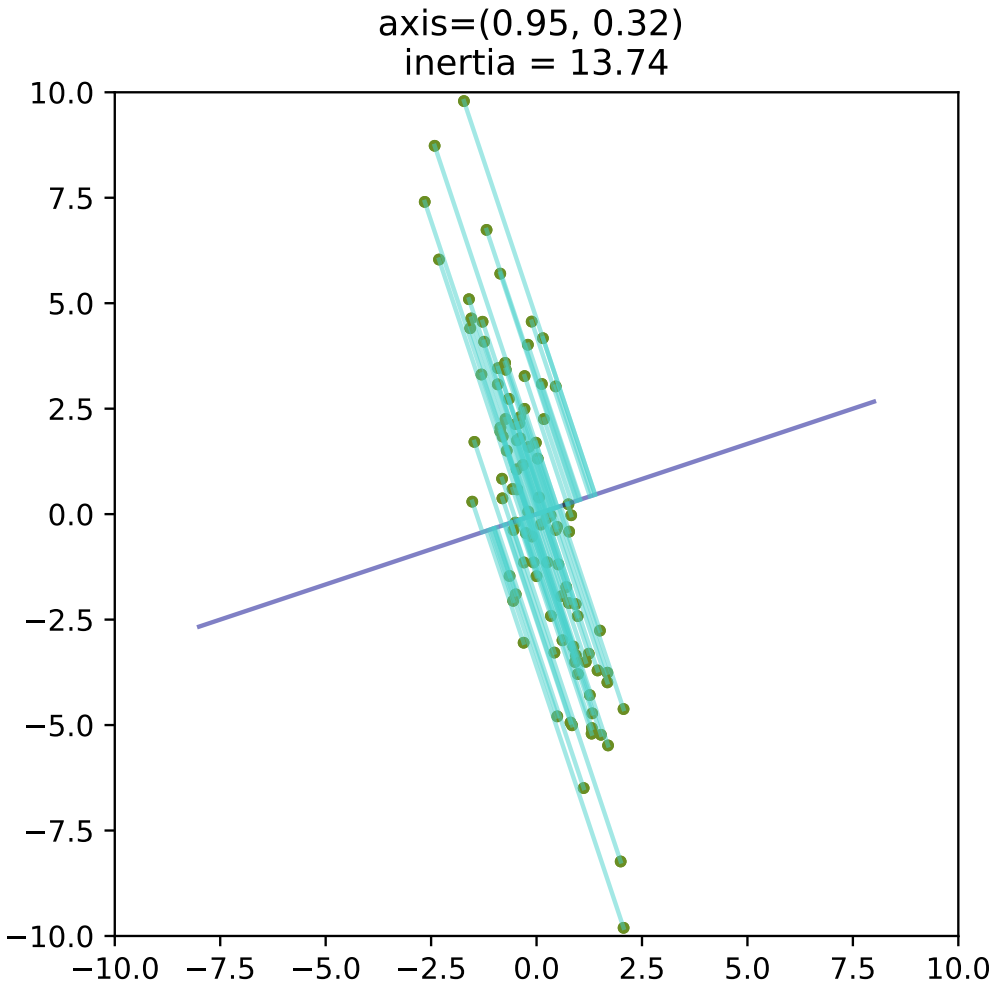
Exercice 2 : **No inertia**

In what situations could we have $I_F = 0$?

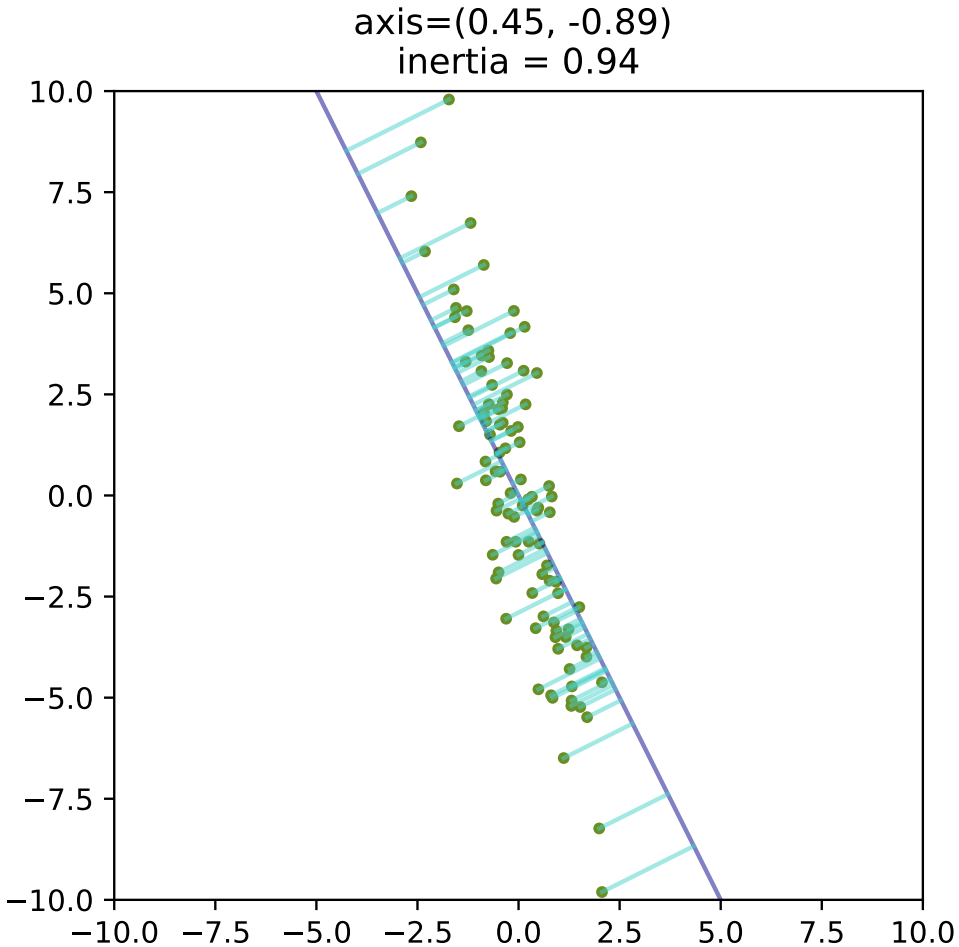
Inertia


axis=(0.05, -1.00)
inertia = 0.74
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0

Inertia


axis=(0.95, 0.32)
inertia = 13.74
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0

Inertia


axis=(0.45, -0.89)
inertia = 0.94
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0

Iris dataset

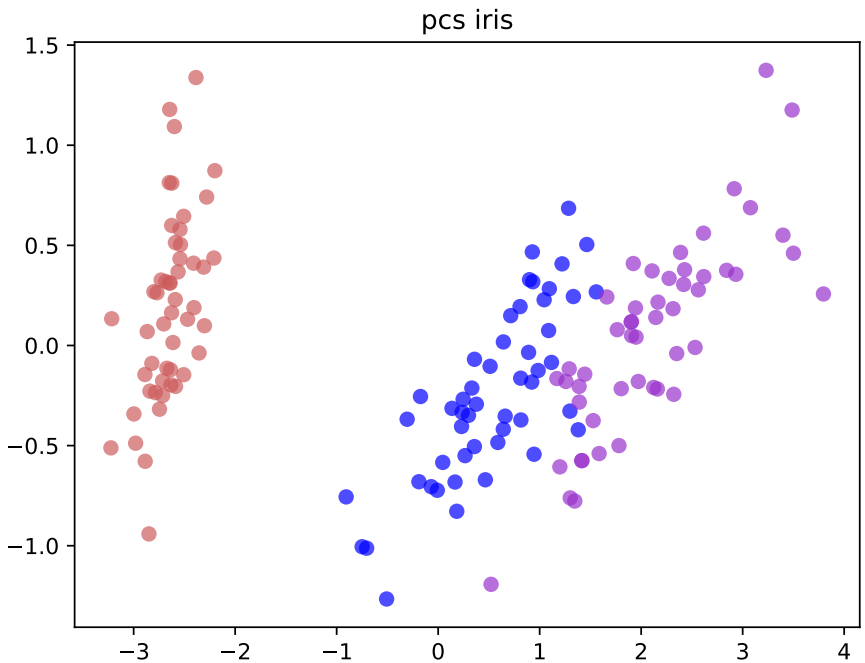


Figure – PCA performed on the iris dataset, keeping 2 dimensions. We see that the principal components are able to separate the data.

In this paper, astrophysicists use PCA in order to test a new star temperature prediction method [?]

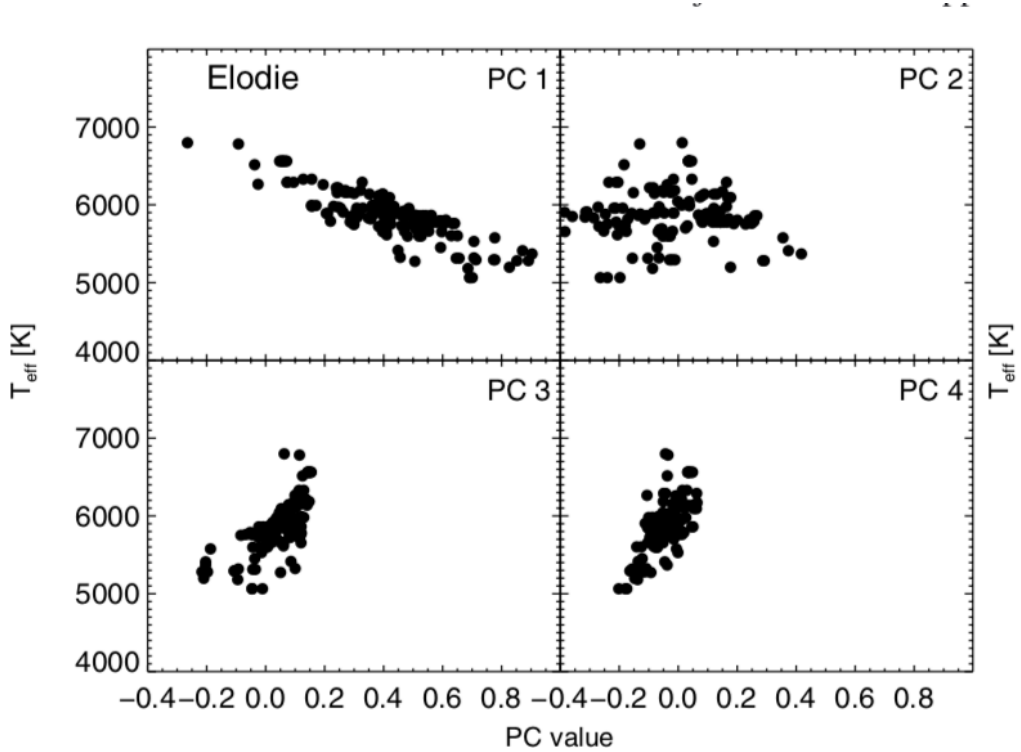
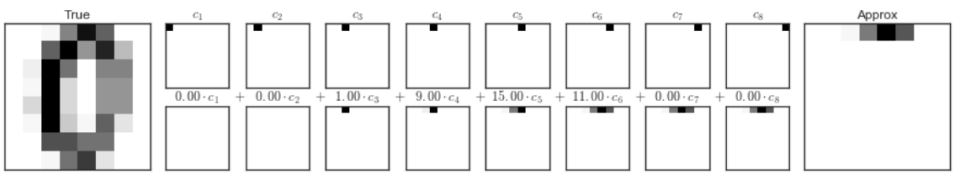


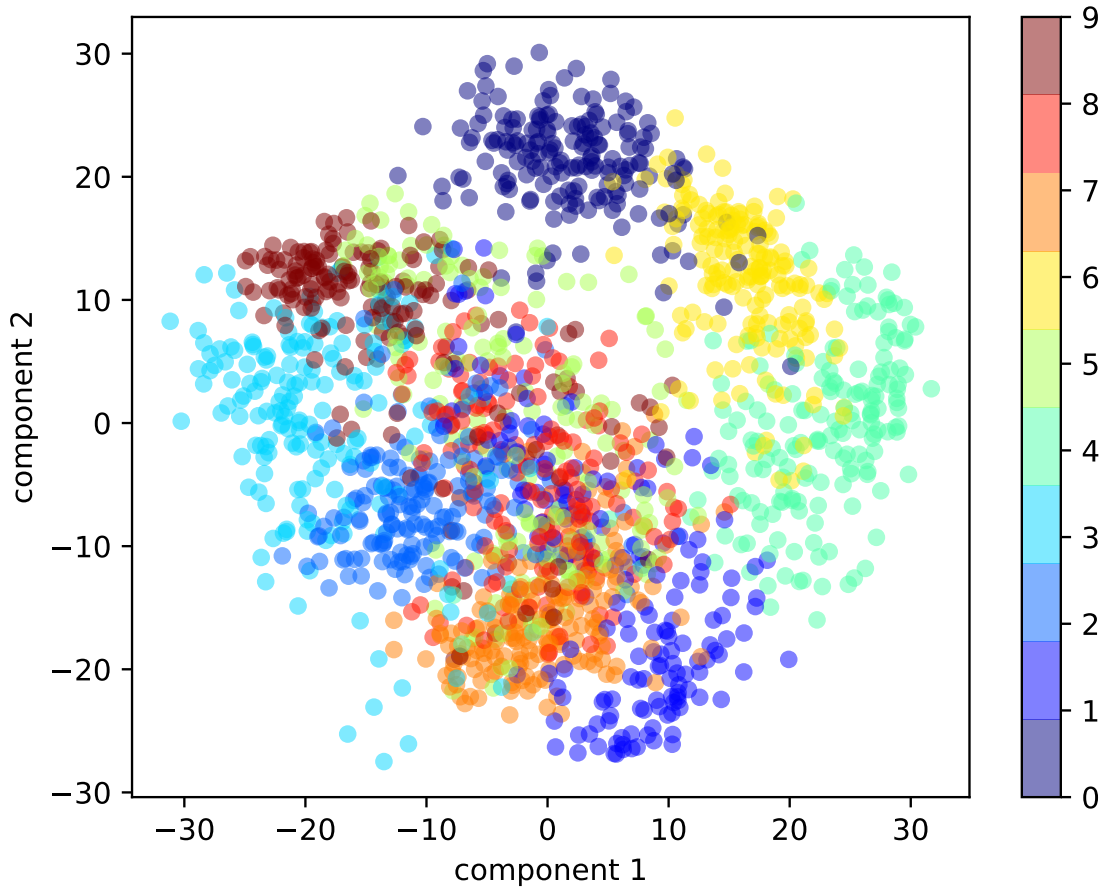
Figure – PCA used in order to predict temperature.

PCA on digits

- We can perform the PCA on a dataset consisting in 8×8 pixels images of digits, in order to see if the PCA allows a visualization of some structure in the data.



PCA on digits



PCA on digits : reconstruction

With 8 principal components, we can monitor the reconstruction of the images (originally in 64 dimensions)

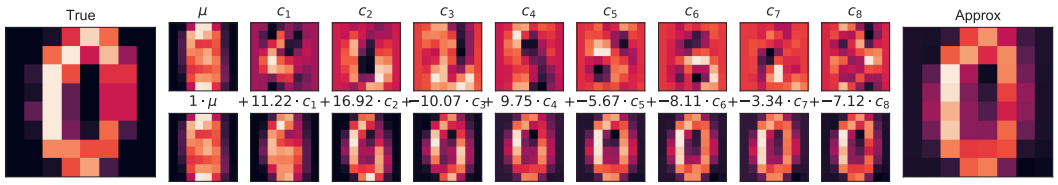


Figure – Reconstruction of 0

`https://jakevdp.github.io/PythonDataScienceHandbook/05.09-principal-component-analysis.html`

PCA on digits : reconstruction

With 8 principal components, we can monitor the reconstruction of the images (originally in 64 dimensions)

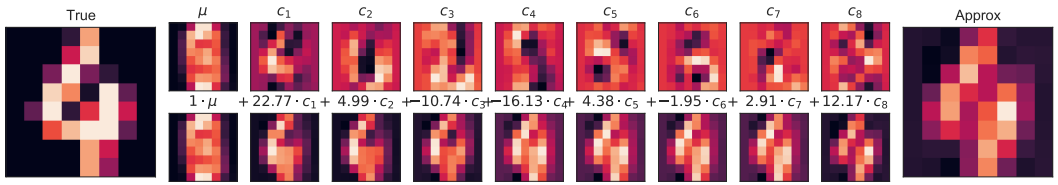


Figure – Reconstruction of 4

`https://jakevdp.github.io/PythonDataScienceHandbook/05.09-principal-component-analysis.html`

Explained variance

A natural question is : what is a relevant number of principal components ?
A common quantity that is used is **explained variance**. Each component w_k carries a percentage of the total variance of the data.

$$\frac{\hat{Var}(w_k^T x)}{\sum_{j=1}^{d} \hat{Var}(x^j)} \tag{14}$$

where $\hat{Var}(x^j)$ is the variance of the (centered) feature j.

$$\hat{Var}(x^j) = \frac{1}{n-1} \sum_{i=1}^{n} (x_i^j)^2 \tag{15}$$

Number of components

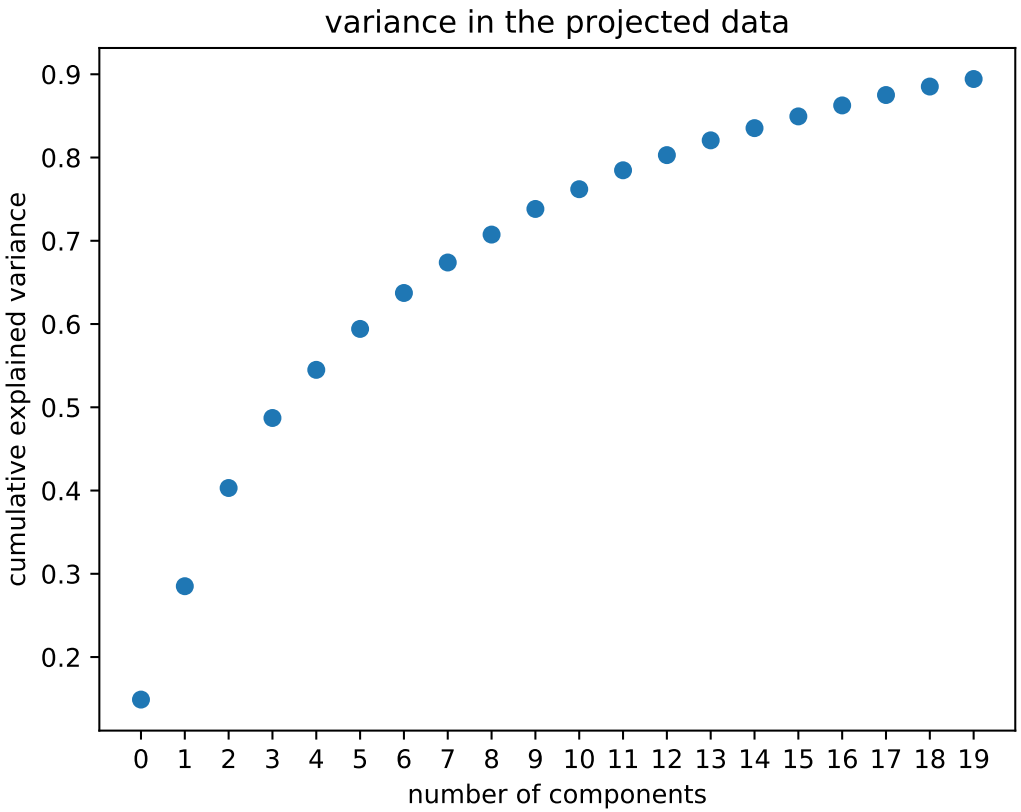
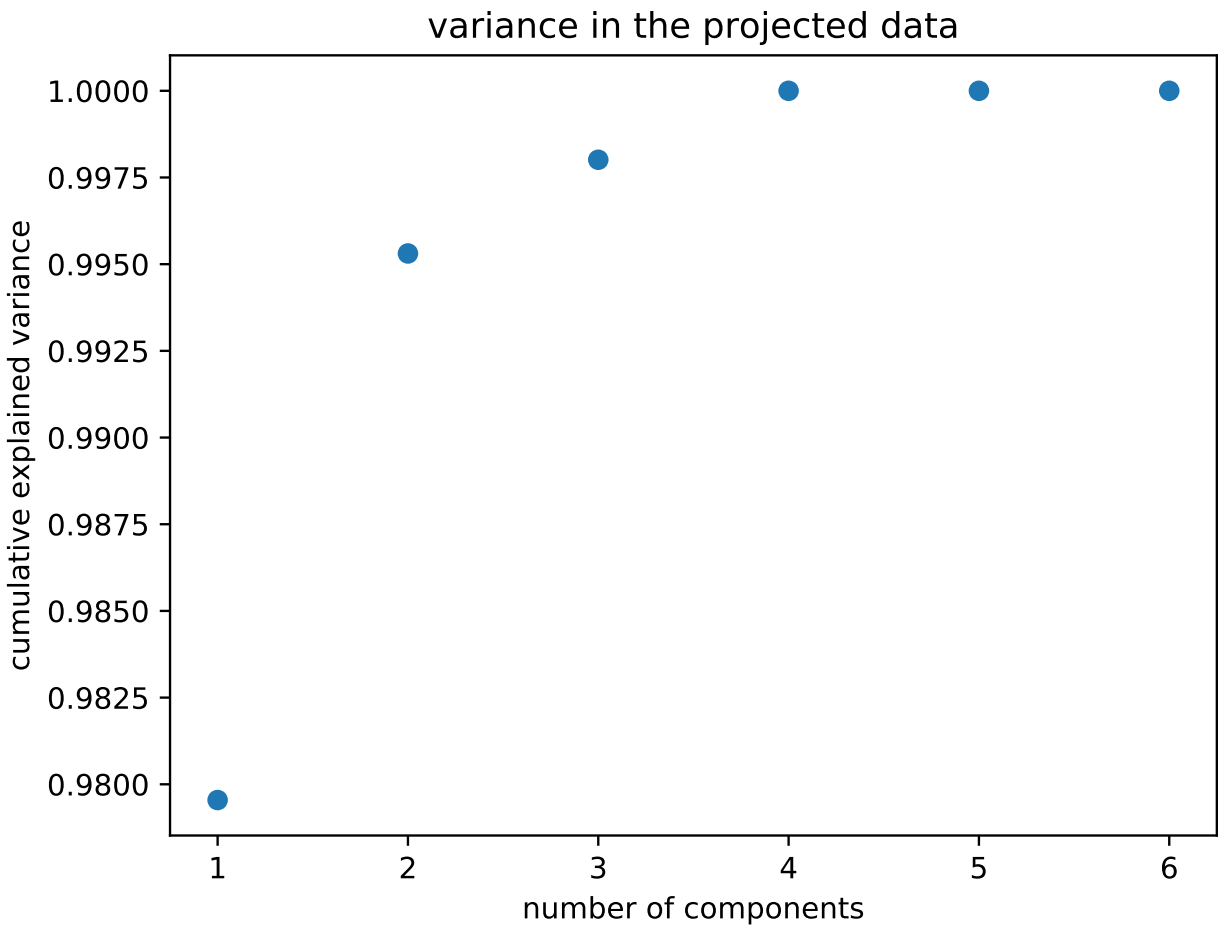


Figure – Variance of the projected data as a function of the number of components (digits dataset)

Number of components

Exercice 3 : What happens with this dataset ?



Number of components

Conclusion : PCA can help detemine whether some components carry no information in the data.

Shortcomings of PCA

PCA is sensitive to :

- outliers
- initial data scaling

Unsupervised learning

From a number of samples x_i, you want to retrieve information on their structure : **modelisation**. The three main unsupervised learning problems are :

- clustering
- density estimation
- dimensionality reduction

Clustering

Clustering consists in partitioning the data. $\forall i, x_i \in \mathcal{X}^n$.

$$D_n = \{(x_i)_{i \in [1,\dots,n]}\} \tag{16}$$

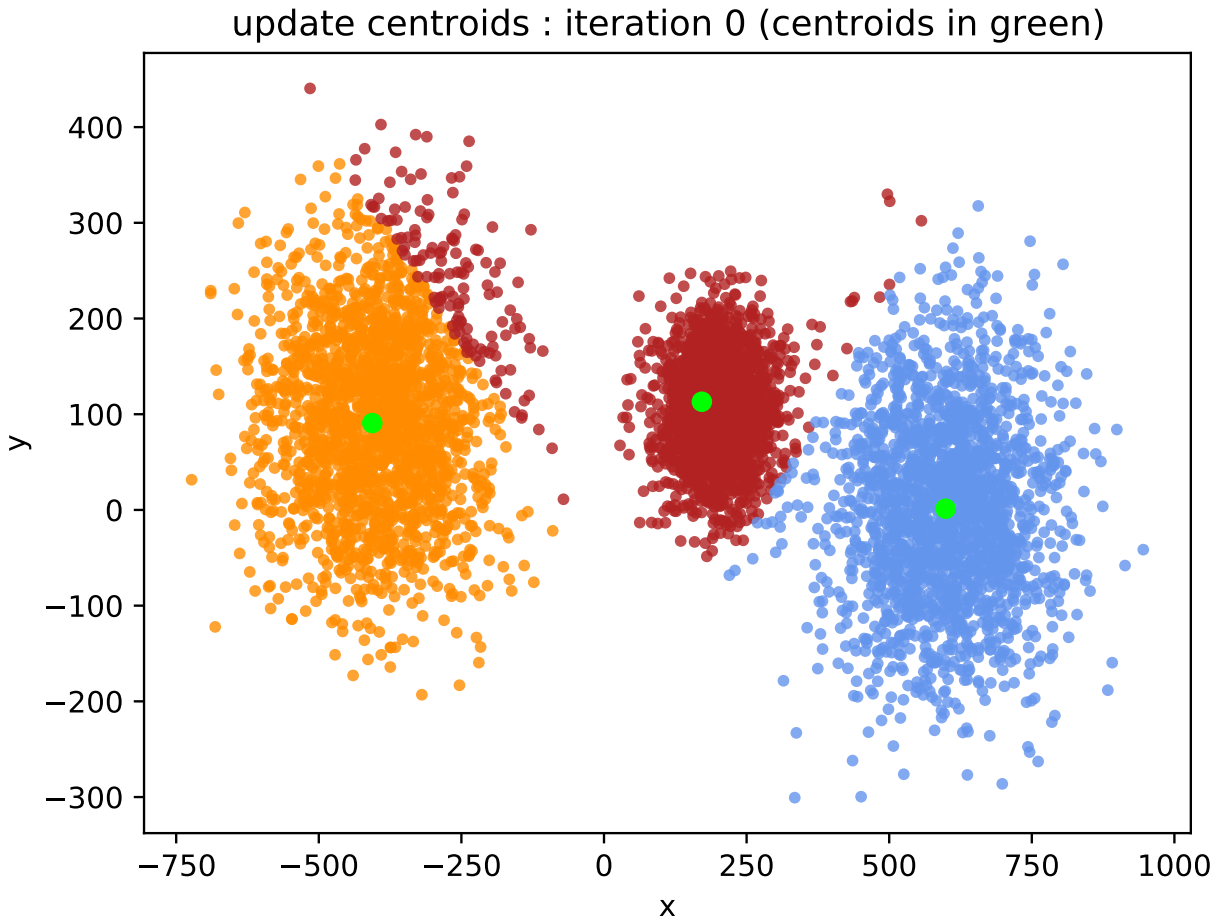
A **partition** is a set of K subsets $A_k \subset D_n$, such that

- $$\cup_{k \in [1,\dots,K]} A_k = D_n \tag{17}$$
- $$\forall k \neq k', A_k \cap A_{k'} = \emptyset \tag{18}$$

Partitions

- **Example 1 :** A is the set of even integers, B the set ot odd integers. Is (A, B) a partition of $\mathbb{N}$?
- **Example 2 :** C is the set of multiples of 2, D the set of multiples of 3. Is (C, D) a partition of $\mathbb{N}$?

Example : partition of data



Applications of clustering

Example applications :

- spam filtering [?,]
- fake news identification [?,]
- marketing and sales
- document analysis [?,]
- traffic classificaiton [?,]

Some of these applications can be considered to be semi-supervised learning.

Vector quantization

Vector quantization consists in computing **prototypes** $\Omega = (\omega_k)_{k\in[1,\dots,K]} \in \mathcal{X}^K$ that represent the data well.
This implies that a **metric** is defined on $\mathcal{X}$.
Most often, this is interesting / useful if $K << n$.

Voronoï subsets

We assume a loss (we can think of it as a distance here) L is defined on $\mathcal{X}$. The **Voronoï subset** of ω is defined as

$$V(\omega) = \{x \in D_n, \arg\min_{\omega' \in \Omega} L(\omega', x) = \omega\} \tag{19}$$

- We assume that arg min returns one single element.
- The Voronoï subsets form a partition of D_n.

Distortion

To measure the quality of a Voronoï partition, we introduce the **distortion** $R(\Omega)$.
For each x, we note $h_\Omega(x) = \arg\min_{\omega' \in \Omega} L(\omega', x)$.

$$R(\Omega) = \frac{1}{n} \sum_{i=1}^{n} L(x_i, h_\Omega(x_i)) \tag{20}$$

Distortion

For each x, we note $h_\Omega(x) = \arg\min_{\omega' \in \Omega} L(\omega', x)$.

$$\begin{aligned} R(\Omega) &= \frac{1}{n} \sum_{i=1}^{n} L(x_i, h_\Omega(x_i)) \\ &= \frac{1}{n} \sum_{\omega \in \Omega} \sum_{x \in V(\omega)} L(x_i, h_\Omega(x_i)) \\ &= \frac{1}{n} \sum_{\omega \in \Omega} V_\Omega(\omega) \end{aligned} \tag{21}$$

with

$$V_\Omega(\omega) = \sum_{x \in V(\omega)} L(x_i, h_\Omega(x_i)) \tag{22}$$

Minimum of distortion

We want to find the prototypes for which the distortion is **minimal**.

- The set of prototypes minimizing distorsion might not be unique.
- We need to tune K (number of prototypes).

Vector quantization techniques

- K-means
- Growing neural gas (GNG)
- Self-organizing maps

K-means clustering

- $\mathcal{X} = \mathbb{R}^d$.
- $L(x, y) = ||x - y||^2$.

Objective function

With

- $\Omega = \{\omega_1, \ldots, \omega_K\} \in \mathbb{R}^{K,d}$.
- $z_i^k = 1$ if x_i is assigned to ω_k, $z_i^k = 0$ otherwise. $z = (z_i^k) \in \mathbb{R}^{n,K}$.

we define the objective function

$$J(\Omega, z) = \sum_{i=1}^{n} \sum_{k=1}^{K} z_i^k ||x_i - \omega_k||^2 \tag{23}$$

It is also called the **inertia.**

K-means algorithm

Result: $\Omega \in \mathbb{R}^{K,d}$
$\Omega \leftarrow$ Random initialization;
$z = M$ where M is the $n \times K$ matrix with 0's;
while *Convergence criteria is not satisfied* **do**
  a] Greedily minimize J with respect to z;
  b] Minimize J with respect to Ω;
end
return Ω

Algorithm 1: K-means (Loyd algorithm)

Stopping criterion

To stop the algorithm, the norm of the difference between Ω_t and Ω_{t+1} must be smaller than a given tolerance. (e.g. $1e^{-4}$). Here it is a norm between matrix (Frobenius norm) :

$$||A||_F = \sqrt{\sum_{i=1}^{n} A_{ij}^2} \tag{24}$$

Minimization

We focus on step b]. How can we minimize J with respect to Ω ?

Minimization

Exercice 4 : **Convexity :**
Show that $J(\Omega, z)$ is convex with respect to Ω.

$$J(\Omega, z) = \sum_{i=1}^{n} \sum_{k=1}^{K} z_i^k ||x_i - \omega_k||^2 \tag{25}$$

Hence, to minimize J with respect to Ω, we just need to cancel the gradient.

Minimization

Exercice 5 : **Gradient :**
Compute the gradient of $J(\Omega, z)$ with respect to Ω and deduce the minimizer Ω^*.

- z is fixed
- we can see Ω has a vector of $\mathbb{R}^{Kd}$.

Convexity

Is $J(\Omega, z)$ convex in z ?

Convexity

Is $J(\Omega, z)$ convex in z ?
No, as z is not even defined on a convex set, so convexity can not be defined anyways !
Hence, the function might have **local minima**.

Suboptimal clustering

Exercice 5 : **Local minima** : propose a setting (dataset, initialization) for which the algorithm outputs a bad set of centroids.

Suboptimal clustering

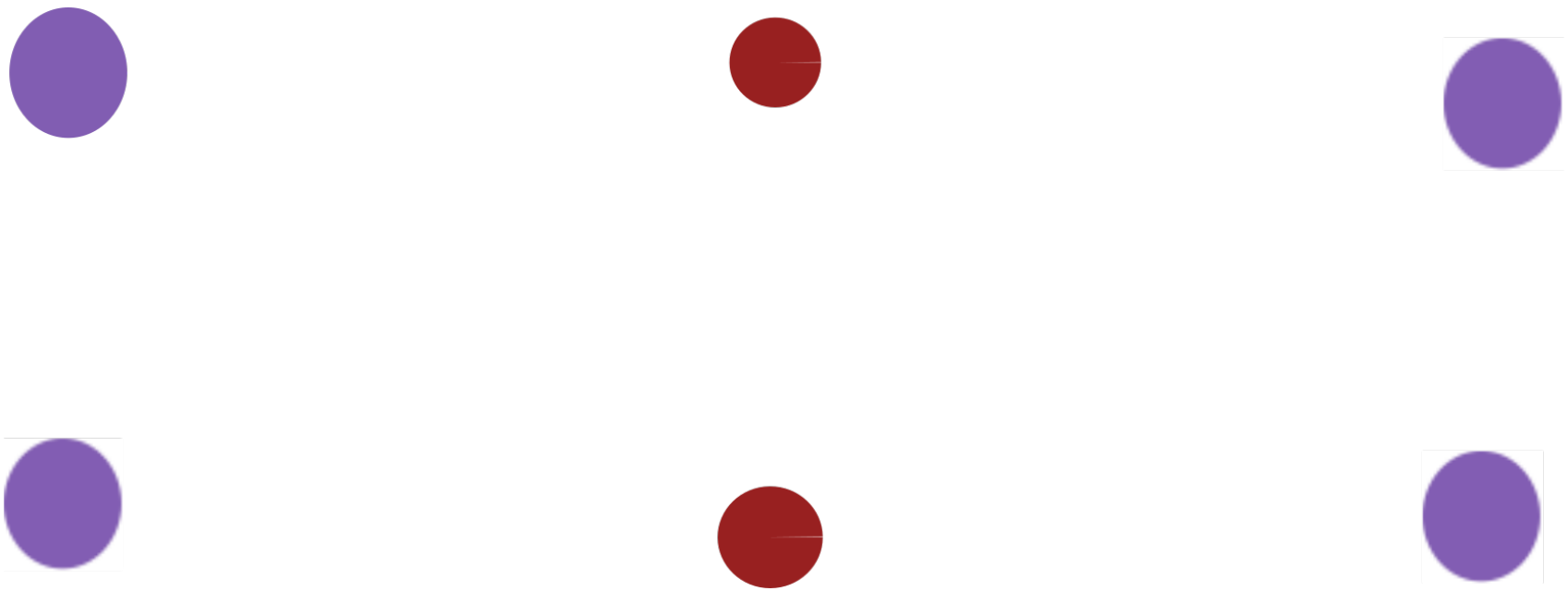


Figure – Initialization of centroids in red.

Random initialization

Hence, the result of K-means strongly depends on the initial position of the centroids.
A common approach is to restart the algorithm several times and select the result with lowest inertia.

Drawbacks of inertia minimization

K-means is based on the minimization of the inertia and hence on the euclidean distance. **However**, in some contexts, the euclidean distance is not the adapted metric.

`https://scikit-learn.org/stable/modules/clustering.html`

References I